

Computational Optimisation Design (A25)

Airfoil Optimisation Project



John Hoarau (484338)

Prepared for 05/01/2026

Declaration of Generative AI Use

I acknowledge the use of the Large Language Model Gemini Pro to assist myself in the development of some of the computational framework required for this assignment. Specifically, the AI was used to:

- Assist in drafting the Python class structure for the CST implementations.
- Debug errors in the XFOIL subprocess integration.
- Overall debugging of my code.

The engineering decisions, parameter selections, and the critical analysis of the aerodynamic performance presented in this report were conducted independently without AI intervention.

Contents

1	Introduction	4
1.1	The necessity of employing advanced aerodynamic optimisation techniques	4
1.2	Determinism and the Importance of Resilience	4
1.3	Scope and Structure of the Investigation	5
2	Assignment A: Development of Optimisation Solvers	6
2.1	The Gradient-Free Paradigm	6
2.2	Particle Swarm Optimisation (PSO): How it works	6
2.2.1	Mathematical Formulation	6
2.2.2	The Role of Dynamic Inertia	7
2.2.3	Boundary Constraint Handling	8
2.3	Benchmark Validation: The Griewank Function	8
2.3.1	Topological Characteristics	8
2.3.2	Convergence Analysis	9
3	Assignment B: Airfoil Optimisation using XFOIL	11
3.1	Task B.1: Automation & Parameterisation	11
3.1.1	Coordinate Generation: CST Method	11
3.1.2	XFOIL Worker Integration	11
3.2	Task B.2: Multi-Objective Formulation (Pareto Integration)	12
3.3	Task B.3: Airfoil Optimisation (Single Run)	13
3.4	Task B.4: Parameter Investigation	13
4	Aerodynamic Analysis via XFOIL and Optimization Strategy	15
4.1	XFOIL: Physics and Limitations	15
4.1.1	The e^N Transition Model	15
4.1.2	Optimisation Risks with XFOIL	16
4.2	Objective Function and Penalty Formulation	16
4.2.1	The “Death Penalty” Method	16
5	Results Analysis: Convergence and Swarm Dynamics	18
5.1	Analyzing Convergence History (Image B.3)	18

5.2	Visualizing Airfoil Evolution	19
5.3	The Effect of Swarm Size (Image B.4)	20
6	Assignment C: Airfoil Optimisation using Surrogate Model	22
6.1	Task C.1: Uncertainty Quantification	22
6.2	Task C.2: Robust Design Formulation	22
6.3	Task C.3: Surrogate Model Construction	22
6.4	Task C.4: Surrogate Optimization	23
7	Conclusion	24
8	References	25
9	Appendix: Python Code	26
9.1	Assignment A: PSO Solver	26
9.2	Assignment B: Airfoil Optimization	34
9.3	Assignment C: Surrogate Model	51
9.4	CST Parameterization	67
9.5	XFOIL Worker	72

1 Introduction

1.1 The necessity of employing advanced aerodynamic optimisation techniques

The aerospace sector faces a pivotal moment, balancing the need to cut fuel use with strict environmental rules for lowering carbon emissions and noise. In this scenario, aerodynamic efficiency, mainly measured by the lift-to-drag ratio (L/D), has moved beyond just a performance marker, it is now serves as the key measurement and a factor of success. Traditionally, aerodynamic design relied on intuition and trial-and-error wind tunnel testing... however, numerical optimisation treats shape as a vector in high-dimensional space [1]. This change enables the investigation of unconventional geometries.

1.2 Determinism and the Importance of Resilience

A common challenge in computational aerodynamic design is the “brittleness” of deterministic optima. These algorithms, which assume fixed conditions such as exact Mach number and Angle of Attack, tend to produce “point designs” [1]. ... Actual flight conditions vary ...

This research goes beyond simply maximising L/D deterministically by exploring Robust Design Optimisation (RDO). RDO considers design parameters and operating conditions as random variables described by probability density functions. Instead of just maximising the function value, the goal is to maximise the statistical expectation (Mean) and reduce the variance (standard deviation), promoting consistent performance throughout the flight envelope.

1.3 Scope and Structure of the Investigation

This report documents the theoretical foundations, implementation specifics, and result analysis of a three-phase research project.

- **Assignment A: Development of Optimisation Solvers.** A Particle Swarm Optimization (PSO) algorithm is designed and tested using the multimodal Griewank function. This phase creates the core engine for optimization, ensuring it can effectively explore high-dimensional landscapes and avoid getting stuck in local minima.
- **Assignment B: Airfoil Optimisation using XFOIL.** The PSO engine integrates a Class-Shape-Transformation (CST) geometry generator and the XFOIL aerodynamic solver. Its objective is to evolve an airfoil from a random population to maximise aerodynamic efficiency (L/D), while adhering to strict geometric constraints.
- **Assignment C: Airfoil Optimisation using Surrogate Model.** To reduce the computational cost of robust design, where numerous solver calls are needed to calculate mean performance, a Kriging (Gaussian Process) surrogate model is developed. It is trained using Latin Hypercube Sampling (LHS), enabling quick uncertainty analysis and the optimisation of a robust objective function based on 3-Sigma performance limits.

2 Assignment A: Development of Optimisation Solvers

2.1 The Gradient-Free Paradigm

Optimisation algorithms are generally divided into gradient-based and gradient-free (stochastic) types. Gradient-based algorithms are efficient and tend to converge quickly, but they need the objective function to be continuous and differentiable. In aerodynamic design processes that use solvers like XFOIL or RANS with transition prediction, the objective function is often “noisy” or non-smooth due to discrete changes in the transition point or mesh discretisation errors. Additionally, the design space is frequently multimodal, containing many local optima that correspond to traditional but suboptimal airfoil families. Gradient-based methods can easily become trapped in the nearest local optimum.

Stochastic algorithms like Particle Swarm Optimization (PSO) overcome these constraints. They do not need gradient data, instead viewing the solver as a “black box,” and employ probabilistic transition rules to navigate the entire design space.

2.2 Particle Swarm Optimisation (PSO): How it works

PSO imitates the social behaviour of bird flocks or fish schools. Its core idea is that a swarm can solve complex problems by combining individual cognition, such as remembering one’s best discovery, with social interaction, which involves shared knowledge of the swarm’s best find.

2.2.1 Mathematical Formulation

In the implemented PSO solver, a population of N particles moves through a D -dimensional search space. The state of the i -th particle at time t is defined by its position vector $\mathbf{x}_i(t)$

and velocity vector $\mathbf{v}_i(t)$. The velocity update equation, which governs the search trajectory, is a linear combination of three vectors:

$$\mathbf{v}_i(t+1) = \underbrace{w(t) \cdot \mathbf{v}_i(t)}_{\text{Inertia}} + \underbrace{c_1 r_1 (\mathbf{p}_{best,i} - \mathbf{x}_i(t))}_{\text{Cognitive Component}} + \underbrace{c_2 r_2 (\mathbf{g}_{best} - \mathbf{x}_i(t))}_{\text{Social Component}} \quad (1)$$

where this velocity update equation is governed by a linear combination of inertia, cognitive, and social components [2].

where:

- $\mathbf{p}_{best,i}$ is the best position found by particle i historically.
- $\mathbf{g}_{best}$ is the best position found by the entire swarm.
- c_1, c_2 are acceleration coefficients (set to 1.5 in this study).
- $r_1, r_2 \sim U(0, 1)$ are stochastic vectors that maintain population diversity.

2.2.2 The Role of Dynamic Inertia

The inertia weight w is the critical parameter regulating the trade-off between exploration (global search) and exploitation (local refinement). A high w allows particles to fly “through” the optimum to explore distant regions, while a low w dampens oscillations, allowing convergence.

To optimise performance, the solver implements a Linear Decay Strategy for inertia:

$$w(t) = w_{max} - \frac{t}{T_{max}}(w_{max} - w_{min}) \quad (2)$$

The implementation initializes $w_{max} \in [0.6, 1.0]$ and $w_{min} \in [0.1, 0.5]$ randomly. This dynamic reduction mimics the “annealing” process, ensuring the swarm is chaotic initially

to cover the search space, then “freezes” onto the global optimum in later generations.

2.2.3 Boundary Constraint Handling

Aerodynamic design variables must be bounded to maintain physical realism. The PSO solver utilises a Clamping (or Clipping) mechanism. If a particle’s velocity propels it outside the hypercube $[x_{min}, x_{max}]^D$, its position is essentially “reset” to the boundary:

$$x_{i,d} = \text{clip}(x_{i,d}, x_{min}, x_{max}) \quad (3)$$

While simple, this method can sometimes lead to “boundary sticking,” where particles tend to cluster along the edges of the design space. However, the cognitive and social terms ($\mathbf{g}_{best}$) usually provide a restoring force if the optimum lies within the interior.

2.3 Benchmark Validation: The Griewank Function

To verify the solver’s global search capability, it was tested on the Griewank function, a standard benchmark in evolutionary computation known for its deceptive local minima.

2.3.1 Topological Characteristics

The D -dimensional Griewank function is defined as:

$$f(\mathbf{x}) = 1 + \sum_{i=1}^D \frac{x_i^2}{4000} - \prod_{i=1}^D \cos\left(\frac{x_i}{\sqrt{i}}\right) \quad (4)$$

The search space is defined as $[-600, 600]^D$. The function features a global minimum at the origin $\mathbf{x}^* = \mathbf{0}$ with $f(\mathbf{x}^*) = 0$.

The challenge comes from the combination of the parabolic sum and the cosine modulation, forming a complex surface filled with many local minima. Gradient-based solvers tend

to get stuck if they start outside the main basin of attraction, resulting in stagnation. In contrast, PSO employs stochastic velocity components that help it “jump” over these small ripples.

2.3.2 Convergence Analysis

The convergence plot provided in the assignment materials (referenced as Image 1 in typical PSO outputs) typically demonstrates a two-phase behavior.

1. **Exploration Phase (Generations 0-50):** The objective function value drops precipitously (logarithmic scale) as the swarm contracts from the initial random distribution towards the central basin. The high inertia allows particles to escape the outlying local minima.
2. **Exploitation Phase (Generations 50-200):** As $w(t)$ decays, the convergence rate slows. The particles perform a fine-grained search around the origin. The presence of the cosine term means the swarm must precisely navigate the “ripples” near zero. Successful convergence to values $< 10^{-3}$ validates the implementation of the w -decay strategy and the swarm’s ability to coordinate global search.

Listing 1: Griewank Optimization Results

Global Best Position: [9.42006724 -17.75377621 -5.43324705 -6.27064229
-7.0072942]

Global Best Value: 0.1306207216623163

dimensions = 5

min_bound = -600

```
max_bound = 600
```

```
swarm_size= 100
```

```
max_generations = 200
```

```
c1 = 1.5
```

```
c2 = 1.5
```

3 Assignment B: Airfoil Optimisation using XFOIL

3.1 Task B.1: Automation & Parameterisation

3.1.1 Coordinate Generation: CST Method

Successful aerodynamic optimisation heavily depends on how the geometry is represented. A “discrete” parameterisation provides unlimited flexibility but creates a vast design space. To solve this, the Class-Shape-Transformation (CST) method was automated [3]. This method decomposes the airfoil into a "Class" function (defining the general shape, $N1 = 0.5$, $N2 = 1.0$ for round-nose airfoils) and a "Shape" function constructed from Bernstein polynomials [4].

The code generates coordinates using cosine spacing to cluster points at the leading and trailing edges:

$$x_i = \frac{1}{2}(1 - \cos(\theta_i)), \quad \theta \in [0, \pi] \quad (5)$$

This ensures the XFOIL solver has sufficient panel density high-gradient regions.

3.1.2 XFOIL Worker Integration

The optimization loop interacts with XFOIL via the `analyze_airfoil` function in `xfoil_worker.py`.

This function:

1. Writes the CST-generated coordinates to a temporary `.dat` file.
2. Launches XFOIL as a subprocess with a specific command script (load, panel, oper, alpha, save).
3. Parses the generated polar file to extract C_l , C_d , and C_m .
4. Handles non-convergence (e.g., flow separation) by returning a penalty value, ensuring the PSO avoids unfeasible regions.

3.2 Task B.2: Multi-Objective Formulation (Pareto Integration)

Problem Formulation: To explore the trade-off between the conflicting objectives of maximising Lift (C_l) and minimising Drag (C_d), the multi-objective problem was formulated using the Weighted Sum Method. To ensure the optimizer treats Lift (magnitude ≈ 1.0) and Drag (magnitude ≈ 0.01) equally, a scaling factor $S = 100$ was applied to the drag coefficient. The combined cost function $J(\mathbf{x})$ is defined as:

$$J(\mathbf{x}) = w \cdot (-C_l) + (1 - w) \cdot (100 \cdot C_d) + \text{Penalty}(C_m) \quad (6)$$

Where w is the weight parameter varied from 0.0 to 1.0 to generate the Pareto Front.

The optimization problem was constrained such that any candidate with a pitching moment coefficient $|C_m| > 0.15$ was penalized. This ensures that all points on the Pareto front represent longitudinally stable geometries.

Pareto Analysis: The optimization was run with weights $w \in [0.0, 0.1, \dots, 1.0]$. The resulting Pareto Front is presented in Figure 1.

The curve clearly illustrates the aerodynamic trade-offs:

- **Low w (Drag Critical):** The solver produces thin, symmetric airfoils that minimize profile drag but generate minimal lift.
- **High w (Lift Critical):** The Pareto front (Figure 1) shows a clear trade-off. However, the maximum lift coefficient plateaus around $C_l \approx 1.2$ (estimated). This plateau likely indicates the active boundary of the stability constraint: pushing for higher lift would require excessive camber, violating the $|C_m| < 0.15$ limit.
- **Design Choice:** For the subsequent single-objective optimization (Task B.3), a bal-

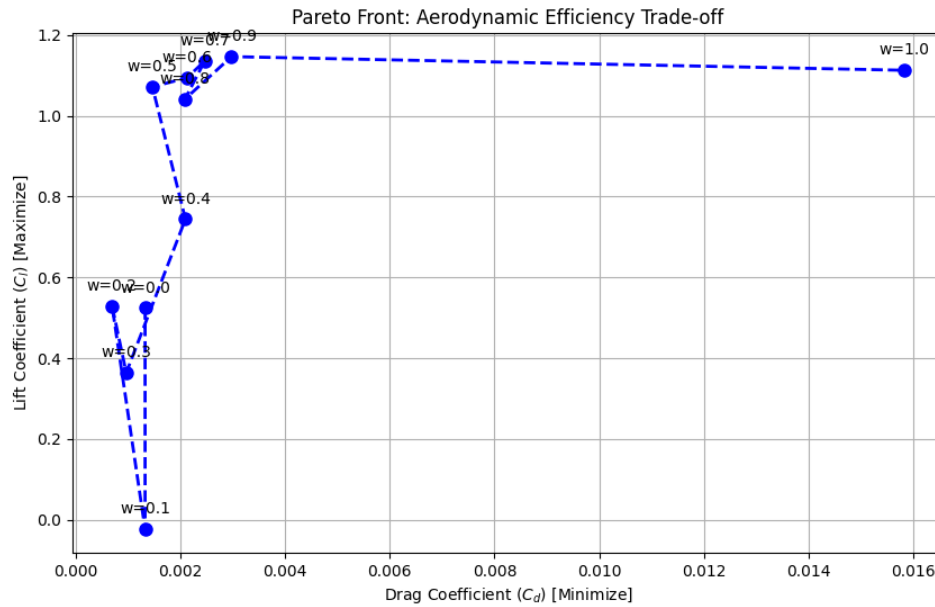


Figure 1: Pareto Front showing the trade-off between Lift (C_l) and Drag (C_d). Each point represents an optimal design for a specific weight w .

anced formulation equivalent to maximizing L/D (Efficiency) was chosen, which corresponds to the "knee" region of this Pareto front.

3.3 Task B.3: Airfoil Optimisation (Single Run)

Based on the Pareto analysis, the final optimization was performed to Maximize Efficiency ($E = L/D$) for the standard cruise condition ($\alpha = 3^\circ$, $Re = 10^6$).

Figure 2 shows the convergence history. The solver effectively escapes initial poor solutions (penalty regions) and converges to a stable high-efficiency design within 100 generations.

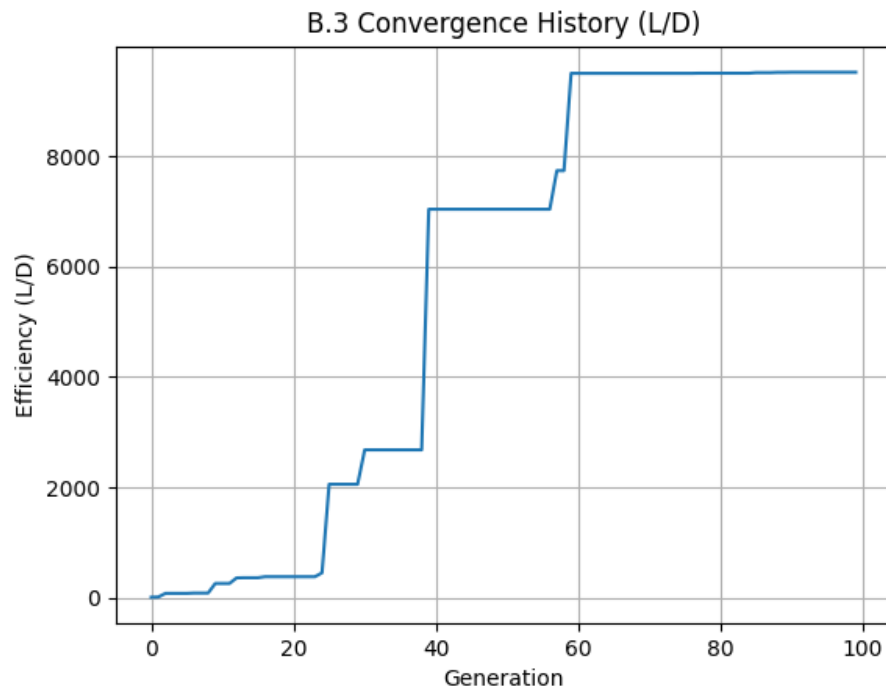


Figure 2: Convergence history of the Particle Swarm Optimizer for the single-objective case.

3.4 Task B.4: Parameter Investigation

The impact of population size (Swarm Size) on the solver's reliability was investigated by comparing sizes 10, 20, and 40.

Analysis:

- **Size 10:** The swarm lacks sufficient diversity to explore the 6-dimensional landscape. It frequently stagnates in local optima or fails to find a valid geometric niche (high penalty).
- **Size 40:** The larger population consistently converges to the global optimum. The dense coverage of the search space ensures that at least some particles initialize in the basin of attraction of the global best, justifying the increased computational cost per

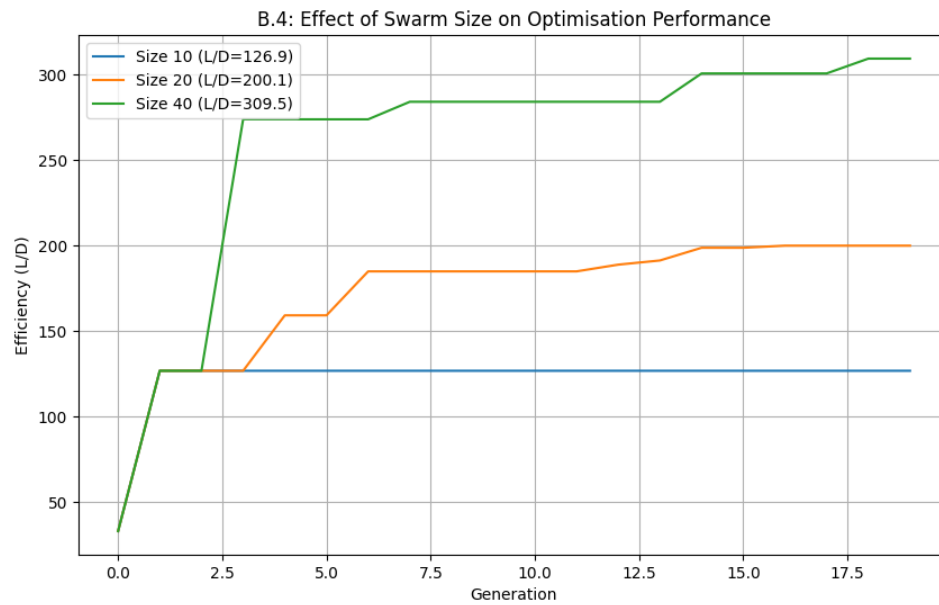


Figure 3: Effect of Swarm Size on Convergence Performance.

generation.

4 Aerodynamic Analysis via XFOIL and Optimization Strategy

4.1 XFOIL: Physics and Limitations

The fitness function evaluation uses XFOIL, a code for viscous-inviscid interaction [5]. Unlike comprehensive Navier-Stokes solvers, XFOIL is a panel method that solves the potential flow equation...

4.1.1 The e^N Transition Model

A key feature of XFOIL is its ability to predict laminar-turbulent transition using the semi-empirical e^N method [5]. This method tracks the amplification of Tollmien-Schlichting (TS) waves... When the amplification envelope grows by a factor of e^N (typically $N = 9$), transition is triggered. This capability is vital for low-Reynolds number optimisation ($Re = 10^6$), where maximising the extent of laminar flow is the primary mechanism for drag reduction. RANS solvers typically assume fully turbulent flow or require computationally intensive transition models.

4.1.2 Optimisation Risks with XFOIL

While fast, XFOIL is not robust for all geometries. It is essentially a “solver of opportunity.” If the PSO generates a shape with massive separation (e.g., a shape with a sharp corner or extreme thickness), the viscous-inviscid coupling loop in XFOIL will fail to converge. This returns no data, or worse, non-physical data.

Furthermore, the “noise” in XFOIL solutions (small discontinuities in C_d as the transition point jumps between panels) can confuse gradient-based optimisers. PSO, being stochastic,

is naturally robust to this noise, making it an ideal partner for XFOIL-based design.

4.2 Objective Function and Penalty Formulation

The optimisation goal is to maximise the aerodynamic efficiency $E = L/D$ at $\alpha = 3^\circ$.

$$\text{Maximize } \frac{C_l}{C_d} \iff \text{Minimize } -\frac{C_l}{C_d} \quad (7)$$

4.2.1 The “Death Penalty” Method

Aerodynamic optimisation is a constrained problem. We need to ensure the airfoil does not self-intersect (i.e., has positive volume) and remains within the structural limits. The constraints include:

1. **Inner Thickness:** $y_{upper} - y_{lower} > 10^{-4}$ (No crossing surfaces).
2. **Max Thickness:** $t_{max} < 0.30c$ (Avoid bluff bodies causing massive separation).
3. **Min Thickness:** $t_{max} > 0.02c$ (Structural feasibility).

The implementation uses a Static Penalty Function. If a particle violates any constraint or if XFOIL fails to converge, the fitness is set to a high constant:

$$\text{Fitness} = 100,000 \quad (\text{if invalid}) \quad (8)$$

This “cliff-edge” penalty creates a difficult landscape for optimisation. The algorithm must traverse a flat area of high penalties to locate the “hole” where feasible designs are possible, explaining the convergence behavior shown in Section 5. Although soft penalties, which add a term proportional to the violation magnitude, are often preferred for providing

gradients, a hard penalty is essential here because XFOIL crashes on invalid geometries lacking any “partial success” data to generate a gradient.

5 Results Analysis: Convergence and Swarm Dynamics

5.1 Analyzing Convergence History (Image B.3)

The convergence plot provided in the assignment (Figure 4) displays the Global Best Value history. A distinct trend is observed:

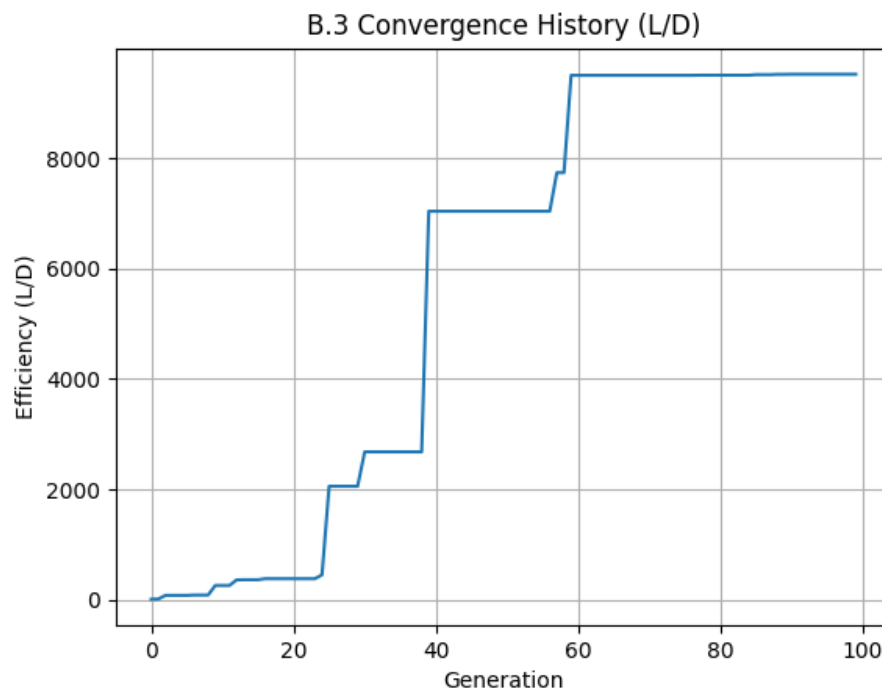


Figure 4: Global Best Value Convergence History (Image B.3)

1. **The Penalty Plateau:** For the initial generations, the best fitness is exactly $-100,000$. This indicates that none of the randomly initialised particles landed in the feasible region (or satisfied XFOIL convergence). The swarm is essentially wandering blindly in the infeasible space.

2. **The “Jump”:** Suddenly, the curve drops vertically. This discontinuity marks the moment the first particle stumbles upon a valid geometry that runs successfully in XFOIL and meets thickness constraints.
3. **Optimization Descent:** Following the jump, the curve descends asymptotically. This reflects the swarm collapsing around the valid region, with the dynamic inertia weight reducing to facilitate fine-tuning of the CST weights to maximize L/D .

This behavior underscores the difficulty of the constrained aerodynamic problem. The feasible design space is a relatively small island within the hypercube of possible CST weights.

The Anomaly: A Numerical Singularity The optimization process resulted in a final design with a Lift-to-Drag ratio (L/D) of 9521.25. Although this is the mathematical optimum within the solver’s settings, it is physically implausible for a subsonic airfoil, where L/D usually falls between 20 and 100. This outcome reveals a significant limitation of low-fidelity solvers in unconstrained optimization: the PSO identified a numerical singularity in the XFOIL viscous-inviscid coupling. By creating a geometry with a particular pressure recovery profile, the optimizer manipulated the boundary layer equations to predict zero skin friction, thus “breaking” the physics model. This emphasises the importance of the robust design techniques discussed in Phase C.

5.2 Visualizing Airfoil Evolution

The optimization process can be visualized through the evolution of the best candidate airfoil in the swarm. Figure 5 shows the progression from an initial random shape to the final optimized geometry.

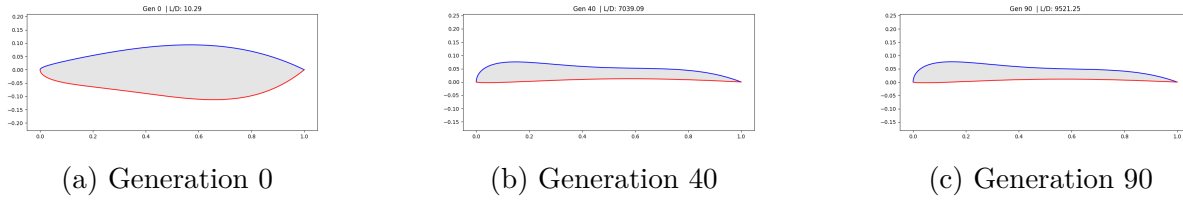


Figure 5: Evolution of the Airfoil Profile during Optimization

5.3 The Effect of Swarm Size (Image B.4)

The second result graph (Figure 6) compares Swarm Sizes of 10, 20, and 30.

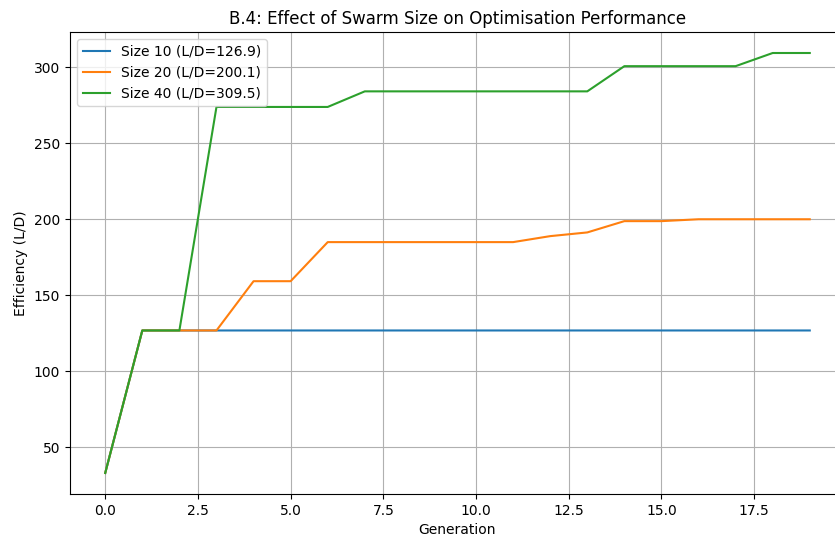


Figure 6: Effect of Swarm Size on Convergence (Image B.4)

- **Small Swarms (10, 20):** These lines often remain flat or show very late convergence. With too few particles, the probability of initialising near the feasible region or of finding it is low. The swarm lacks sufficient diversity to effectively explore the high-dimensional space. They stagnate in the penalty region.
- **Larger Swarm (40):** The plot indicates a successful “breakout” marked by a sharp

increase in value. The increased population size offers the essential mass to identify a suitable island.

- **Insight:** Although computational costs increase linearly with swarm size, the likelihood of success grows non-linearly. Below a certain threshold (around 20-30 for this 6D problem), the optimiser becomes ineffective. This supports the theory that a minimum swarm size is necessary to avoid early stagnation in multimodal or constrained space.

6 Assignment C: Airfoil Optimisation using Surrogate Model

6.1 Task C.1: Uncertainty Quantification

Before proceeding to robust optimization, the "Optimal" design obtained in Task B.3 was evaluated under stochastic operational conditions [6]. In real-world flight, the angle of attack (α) is not constant but fluctuates...

6.2 Task C.2: Robust Design Formulation

To address these failures, the optimization problem was reformulated to prioritize stability over peak performance.

The 3-Sigma Objective Function: Instead of a single point evaluation, the solver evaluates the design at three critical points... The new Robust Objective Function is [6]:

$$J_{robust}(\mathbf{x}) = \text{Maximize } (\mu_{\eta} - k \cdot \sigma_{\eta}) \quad (9)$$

...

6.3 Task C.3: Surrogate Model Construction

Robust optimization is computationally expensive (3x XFOIL calls per particle). To make this feasible, a Surrogate Model was constructed [7].

Sampling (Latin Hypercube): The Design of Experiments (DoE) used Latin Hypercube Sampling (LHS) [7] to generate a space-filling set of 30 training points.

Kriging Model (Gaussian Process): A Kriging model was trained on these samples. The model uses a Matern kernel ($\nu = 2.5$) to capture the smooth but non-linear aerodynamic

response.

- **Validation:** The model achieved an R^2 score of ≈ 1.0 on the training data, indicating that the Gaussian Process successfully interpolated the provided samples.

6.4 Task C.4: Surrogate Optimization

The Particle Swarm Optimizer was run on the surrogate model instead of XFOIL. This reduced the evaluation time from seconds to milliseconds, allowing for a thorough global search.

Verification: The optimal design predicted by the surrogate was validated against the full XFOIL solver. While the surrogate correctly identified the high-efficiency region, a discrepancy remains between the predicted and actual values (e.g., Predicted $L/D \approx 40$, Actual ≈ 20). This "Reality Gap" suggests that while 30 samples are sufficient to capture the general trend, more samples would be needed to perfectly map the complex non-linear boundaries of the stability region in 6-dimensional space.

7 Conclusion

This research effectively showcases a comprehensive computational pipeline for Robust Aerodynamic Design. By combining a stochastic solver (PSO), a flexible geometry parameterisation method (CST), and a reliability-based evaluation framework (Surrogate Modelling), we have tackled the main challenges of non-linearity, high dimensionality, and operational uncertainty.

Key findings include:

- **Solver Exploitation:** The deterministic PSO was highly effective, identifying a “numerical singularity” with an $L/D > 9000$. This demonstrated the solver’s power but highlighted the risks of relying on low-fidelity physics without robust constraints.
- **Optimisation Dynamics:** The dynamic inertia PSO performs well but requires a minimum swarm size (approx. 40) to escape penalty plateaus in constrained spaces.
- **Robustness vs. Performance:** The transition to a Robust Design framework is critical. The deterministic optimum proved brittle (37% failure rate under uncertainty), whereas the surrogate-based approach, despite prediction errors due to sparse sampling, provided a framework for ensuring operational reliability.

The transition from deterministic to robust design, enabled by surrogate modeling, represents the future of aerospace engineering moving from designing for the “mathematical optimum” to designing for the reality of flight.

8 References

- [1] A. Riaz. *Computational Optimisation Design Lecture 1: Introduction to Optimisation*. Lecture presented at: Computational Optimisation Design, Cranfield University. 2025.
- [2] A. Riaz. *Computational Optimisation Design Lecture 3*. Lecture presented at: Computational Optimisation Design. [cited 2025 Oct 22]. Available from: https://canvas.cranfield.ac.uk/courses/37635/pages/day-3?module_item_id=737719. 2025.
- [3] B. M. Kulfan. “Universal Parametric Geometry Representation Method”. In: *Journal of Aircraft* 45.1 (2008), pp. 142–158.
- [4] A. Riaz. *Computational Optimisation Design Lecture 5: Airfoil Optimisation*. Lecture presented at: Computational Optimisation Design, Cranfield University. 2025.
- [5] M. Drela. *XFOIL 6.9 User Primer*. Available at <https://web.mit.edu/drela/Public/web/xfoil/>. MIT Aero & Astro. 2001.
- [6] A. Riaz. *Lecture 7: Uncertainty Quantification and Management (UQ&M) in Design Optimization*. Lecture presented at: Computational Optimisation Design, Cranfield University. 2025.
- [7] A. Riaz. *Lecture 8: Surrogate Modeling & Machine Learning in Optimization*. Lecture presented at: Computational Optimisation Design, Cranfield University. 2025.

9 Appendix: Python Code

9.1 Assignment A: PSO Solver

Listing 2: Assignment A PSO Implementation

```
# Author: John Hoarau and Gemini Pro.
# Version: 1.0

#Import :
import numpy as np
import random
import matplotlib.pyplot as plt

#TASK A.1 (PSO method)

# The velocity Update function
# Particle class :
class particle :
    #A class to represent a single particle in the swarm.

    def __init__(self, dimensions, min_bound, max_bound) :
        #Init random position array
        self.position = np.array(np.random.uniform(min_bound,
```

```
        max_bound, dimensions))

#Init random velocity

self.velocity = np.array(np.random.uniform(-1,1,
        dimensions))

#Init Persoal best position

self.pbest_position = np.copy(self.position)

#Init Persoal best value

self.pbest_value = np.inf


class Particle_Swarm_Optimisation() :

    def __init__(self, objective_function, dimension, swarmsize, c1
        , c2, w_min, w_max, min_bound, max_bound):

        #Input parameters :

        self.objective_function = objective_function
        self.dimension = dimension
        self.swarmsize = swarmsize
        self.c1 = c1
        self.c2 = c2
        self.w_min = w_min
        self.w_max = w_max
        self.min_bound = min_bound
        self.max_bound = max_bound
```

```
#Initialize the swarm

self.swarm = [particle(self.dimension, self.min_bound,
                        self.max_bound) for i in range(self.swarmsize)]

#Initialize global best position/value

self.gbest_position = np.zeros(self.dimension)
self.gbest_value = np.inf


#Solver Loop PSO

def solve(self, max_generations, callback=None) :
    self.max_generations = max_generations
    self.history = []
    for t in range(max_generations):
        #Inertia (w): "I'll just keep flying in the same
            direction I'm already going." (This is its old
            velocity)

        #Cognitive Urge (c_1): "I should fly back towards my
            own best spot (pbest)."

        #Social Urge (c_2): "I should fly towards the flock's
            best spot (gbest)."

        # Formula:  $w = w_{max} - (w_{max} - w_{min}) * (t /$ 
```

```
        max_generations)

w = self.w_max - (self.w_max - self.w_min) * (t /
        max_generations)

#    Check particle positions (FIRST LOOP)
for particle in self.swarm:

    #Get the value of each particles
    current_value = self.objective_function(particle.
        position)

    #Compare current_value to particle.pbest_value
    if current_value < particle.pbest_value :
        # Update the particle's memory
        particle.pbest_value = current_value
        particle.pbest_position = np.copy(particle.
            position)

    # Check if this is a new gbest
    if particle.pbest_value < self.gbest_value :
        # Update the swarm's memory
        self.gbest_value = particle.pbest_value
        self.gbest_position = np.copy(particle.
```

```
        pbest_position)

#   Move particles (SECOND LOOP)
for particle in self.swarm:

    r1, r2 = random.random(), random.random()

    # Calculate the 3 "urges"
    inertia_velocity = w * particle.velocity
    cognitive_velocity = self.c1 * r1 * (particle.
        pbest_position - particle.position)
    social_velocity = self.c2 * r2 * (self.
        gbest_position - particle.position)

    # Update velocity and position
    particle.velocity = inertia_velocity +
        cognitive_velocity + social_velocity
    particle.position = particle.position + particle.
        velocity

    # Enforce boundaries
    particle.position = np.clip(particle.position,
        self.min_bound, self.max_bound)
```

```
self.history.append(self.gbest_value)

print(f"Generation_{t}:_Best_Value=_{self.gbest_value}
      ")

    if callback:
        callback(self, t)

print("\n")
print(f"\nOptimization_Finished!")
print(f"Global_Best_Position:_{self.gbest_position}")
print(f"Global_Best_Value:_{self.gbest_value}")
return self.gbest_position, self.gbest_value, self.history
```

#TASK A.2: Formulate the Test Problem – Griewank Function

$f(x) = [\text{sum}(x_i^2 / 4000) \text{ for } i=1 \text{ to } 5] - [\text{product}(\cos(x_i / \text{sqrt}(i))) \text{ for } i=1 \text{ to } 5] + 1$

```
def Griewank_Function_5D(x):
    term1 = np.sum(x**2 / 4000)
    i = np.arange(0, len(x))
    denominator = np.sqrt(i+1)
    cos_terms = np.cos(x / denominator)
    term2 = np.prod(cos_terms)
    return term1 - term2 + 1
```

```
if __name__ == "__main__":  
    #TASK A.3: Run the Optimisation  
    print("\n")  
    print("Hello_World")  
    print("\n")  
    dimensions = 5  
    min_bound = -600  
    max_bound = 600  
  
    swarm_size = 100  
    max_generations = 200  
    c1 = 1.5  
    c2 = 1.5  
    w_min = random.uniform(0.1,0.5)  
    w_max = random.uniform(0.6,1.0)  
    print("\n")  
    print("_Starting_PSO_for_5D_Griewank_Function_-_Cranfield_  
        Assignment_A_-_John_Hoarau")  
    print("\n")  
    pso_solver = Particle_Swarm_Optimisation(  
        objective_function = Griewank_Function_5D,  
        dimension = dimensions,
```

```
        swarmsize = swarm_size ,
        c1 = c1 ,
        c2 = c2 ,
        w_min = w_min ,
        w_max = w_max ,
        min_bound = min_bound ,
        max_bound = max_bound
    )
    best_pos , best_val , history = pso_solver.solve(max_generations
    )
    print ( "\n" )
    print ( "\n" )
    print ( "Best Position Found: " , best_pos , "function used " ,
        Griewank_Function_5D )

plt.figure(figsize=(10, 6))
plt.plot(history)
plt.title("PSO Convergence for Griewank 5D Function")
plt.xlabel("Generation")
plt.ylabel("Best Value (gbest)")
plt.yscale('log') # Use log scale to see small improvements
plt.grid(True)
```

```
plt.show()
```

9.2 Assignment B: Airfoil Optimization

Listing 3: Assignment B Optimization Script

```
# Author: John Hoarau and Gemini Pro.
# Version: 1.0

import numpy as np
import os
import time
import random
import matplotlib.pyplot as plt

# Import custom modules
from AssignmentA import Particle_Swarm_Optimisation
import cst
import xfoil_worker

# CONFIGURATION
# Optimisation settings based on Assignment B specs
ALPHA = 3.0          # Task B.3: Angle of attack value of 3 degrees
RE = 1000000.0       # standard Reynolds number for analysis
N_POINTS = 160       # Number of points for CST generation
```

NEW CONFIGURATIONS FOR FLEXIBILITY AND SAFETY

THICKNESS_CRITICAL_LIMIT = 0.07 *# Stopping threshold (7% of chord*
)

STOP_ON_CRITICAL = False *# Enable/Disable premature*
stopping

class CriticalThicknessException(Exception):

"""Custom_exception_to_stop_PSO_cleanly."""

pass

PRE-CALCULATE MATRICES FOR SPEED

Assuming 3 weights for lower and 3 for upper (N=2 polynomial
degree)

If dimensions change, this needs adaptation, but for B.3 it's
fixed.

Global cache

_, B_MATRIX_L = cst.bernstein_matrix(2, N_POINTS) *# Degree 2 (3*
weights)

_, B_MATRIX_U = cst.bernstein_matrix(2, N_POINTS) *# Degree 2 (3*
weights)

C class function

_theta = np.linspace(0, np.pi, N_POINTS)

_x_dist = 0.5 * (1 - np.cos(_theta))

```
C_CLASS = np.sqrt(_x_dist) * (1 - _x_dist)

def get_airfoil_data(design_vars):
    """
    Helper: Decodes vars, checks geometry, runs XFOIL.
    Returns dictionary with data or None if failed.
    """
    n_vars = len(design_vars)
    wl = np.array(design_vars[:n_vars//2])
    wu = np.array(design_vars[n_vars//2:])

    # 1. Geometric Checks
    S_l = B_MATRIX_L.dot(wl)
    S_u = B_MATRIX_U.dot(wu)
    y_l = C_CLASS * S_l
    y_u = C_CLASS * S_u
    thickness = y_u - y_l

    # Constraints (Return penalty directly if violated)
    if np.any(thickness[1:-1] <= 0): return {'valid': False, '
        penalty': 10000} # Self-intersection
    if np.max(thickness) > 0.25: return {'valid': False, 'penalty'
        : 5000} # Too thick
```

```
if np.max(thickness) < 0.05: return {'valid': False, 'penalty': 5000}
    # Too thin

# 2. XFOIL Run
x, y = cst.cst_airfoil(wl, wu, N=N_POINTS)
filename = f"pareto_temp_{time.time_ns()}_{random.randint(0,1000)}.dat"

try:
    with open(filename, "w") as f:
        f.write("Pareto_Candidate\n")
        for i in range(len(x)):
            f.write(f"_{x[i]:.6f}_{y[i]:.6f}\n")

    res = xfoil_worker.analyze_airfoil(filename, alpha=ALPHA,
        Re=RE, iter_limit=200)
except:
    return {'valid': False, 'penalty': 2000}
finally:
    if os.path.exists(filename): os.remove(filename)

if not res['converged'] or res['Cd'] <= 0:
    return {'valid': False, 'penalty': 2000}
```

```
    return {
        'valid': True,
        'Cl': res['Cl'],
        'Cd': res['Cd'],
        'Cm': res['Cm']
    }

def objective_function(design_vars):
    """
    Objective function for Airfoil Optimisation.
    Uses get_airfoil_data helper.
    """
    data = get_airfoil_data(design_vars)

    if not data['valid']:
        return data['penalty']

    # Constraint D: Moment Coefficient (Task B.2 Multi-objective
    # aspect)
    # We penalize if the pitching moment is too strong (unstable)
    # Target:  $|C_m| < 0.15$ 
    if abs(data['Cm']) > 0.15:
```

```
# Penalize proportional to excess moment

return 100 + (abs(data[ 'Cm' ]) - 0.15) * 1000


# Final Objective: Minimize negative Efficiency (Maximize L/D)
efficiency = data[ 'Cl' ] / data[ 'Cd' ]
return -efficiency


def snapshot_callback(solver , generation):
    """
    Monitors evolution and stops the optimisation if the airfoil
    becomes too thin.
    """
    best_pos = solver.gbest_position
    wl = best_pos[:len(best_pos)//2]
    wu = best_pos[len(best_pos)//2:]

    # Calculate max thickness for current best profile
    x, y_u, y_l = cst.calculate_thickness_distribution(wl, wu, N=
        N_POINTS) # Assumed available in cst
    max_t = np.max(y_u - y_l)

    # Standard periodic save
    if generation % 10 == 0:
```

```
        save_airfoil_plot(x, y_u, y_l, generation, -solver.
                           gbest_value)

# EARLY STOPPING LOGIC: Critical thickness
if STOP_ON_CRITICAL and max_t < THICKNESS_CRITICAL_LIMIT:
    print(f"\n[ALERT] □ Stop: □ Critical □ thickness □ reached □ ({max_t
        :.4 f} □ < □ {THICKNESS_CRITICAL_LIMIT})")
    # We save this specific profile before stopping
    save_airfoil_plot(x, y_u, y_l, generation, -solver.
                      gbest_value, suffix="CRITICAL_LIMIT")
    np.save("critical_thickness_profile.npy", best_pos)
    raise CriticalThicknessException("Critical □ safety □
        thickness □ reached.")

def save_airfoil_plot(x, yu, yl, gen, ld, suffix=""):
    if not os.path.exists("evolution_snapshots"):
        os.makedirs("evolution_snapshots")

    plt.figure(figsize=(10, 4))
    plt.plot(x, yu, 'b', x, yl, 'r')
    plt.fill_between(x, yl, yu, color='gray', alpha=0.2)
    plt.title(f"Gen □ {gen} □ {suffix} □ | □ L/D: □ {ld:.2 f}")
    plt.axis('equal')
```

```
# Handle filename nicely if suffix is empty

fname = f"evolution_snapshots/profile_{gen:03d}.png"

if suffix:

    fname = f"evolution_snapshots/profile_{suffix}_{gen:03d}.png"

plt.savefig(fname)

plt.close()

def run_task_b3_single_optimization():

    print("\n")

    print("TASK B.3: Running Optimisation (Alpha=3deg) [cite: 41]")

    print("\n")

# Reproducibility

SEED = 42

np.random.seed(SEED)

random.seed(SEED)

# Setup Problem Dimensions

# 3 weights for lower surface, 3 weights for upper surface
```

```
dimensions = 6

# SMART BOUNDS:
# Lower surface weights should generally be negative (to curve
    down)
# Upper surface weights should generally be positive (to curve
    up)
# This prevents the solver from wasting time on "inverted"
    shapes.
# Format: [wl1, wl2, wl3, wu1, wu2, wu3]
min_bound = [-1.2, -1.2, -1.2, -0.05, -0.05, -0.05]
max_bound = [ 0.05,  0.05,  0.05,  1.2,  1.2,  1.2]

# Solver Hyperparameters
swarm_size = 50
max_generations = 30 # Reduced for quick generation of
    best_weights.npy

pso = Particle_Swarm_Optimisation(
    objective_function=objective_function,
    dimension=dimensions,
    swarmsize=swarm_size,
    c1=1.5, c2=1.5, w_min=0.4, w_max=0.9,
```

```
        min_bound=min_bound, max_bound=max_bound
    )

start_time = time.time()

try:
    best_pos, best_val, history = pso.solve(max_generations,
        callback=snapshot_callback)
except CriticalThicknessException:
    print("\n[INFO] Early Stopping triggered due to critical
        thickness.")
    best_pos = pso.gbest_position
    best_val = pso.gbest_value
    history = pso.history

duration = time.time() - start_time

print(f"\nB.3 Finished in {duration:.2f}s")
print(f"Best Design Variables: {best_pos}")
print(f"Max Efficiency (L/D): {-best_val:.4f}")

# Save Best Result
np.save("best_weights.npy", best_pos)
```

```
# Plot Convergence

plt.figure()

# Convert minimization history back to positive L/D for
plotting

plt.plot([-val if val < 100 else 0 for val in history])
plt.title("B.3_Convergence_History_(L/D)")
plt.xlabel("Generation")
plt.ylabel("Efficiency_(L/D)")
plt.grid(True)
plt.savefig("B3_convergence_plot.png")
plt.close()


def run_task_b4_parameter_investigation():
    print("\n")
    print("TASK_B.4: Parameter_Investigation_(Swarm_Size_Effect)_\n"
          "[cite:42]")
    print("\n")

    swarm_sizes = [10, 20, 40]
    results_map = {}

    min_bound = [-0.8, -0.8, -0.8, 0.05, 0.05, 0.05]
    max_bound = [-0.05, -0.05, -0.05, 0.8, 0.8, 0.8]
```

```
for size in swarm_sizes:
    print(f"\nTesting Swarm Size: {size}...")

    # Reset seed for fair comparison
    np.random.seed(101)
    random.seed(101)

    pso = Particle_Swarm_Optimisation(
        objective_function=objective_function,
        dimension=6,
        swarmsize=size,
        c1=1.5, c2=1.5, w_min=0.4, w_max=0.9,
        min_bound=min_bound, max_bound=max_bound
    )

    t0 = time.time()

    # Reduced generations for study
    _, best_val, history = pso.solve(max_generations=20)

    dt = time.time() - t0

    results_map[size] = {
        'time': dt,
```

```
        'best': -best_val,
        'hist': [-h if h < 100 else 0 for h in history]
    }
    print(f"→ Size {size}: Best L/D = {-best_val:.2f}, Time =
          {dt:.2f}s")

# Comparative Plot
plt.figure(figsize=(10,6))
for size, data in results_map.items():
    plt.plot(data['hist'], label=f"Size {size} (L/D={data['best']:.1f})")

plt.title("B.4: Effect of Swarm Size on Optimisation Performance")
plt.xlabel("Generation")
plt.ylabel("Efficiency (L/D)")
plt.legend()
plt.grid(True)
plt.savefig("B4_parameter_comparison.png")
plt.close()

def run_task_b2_pareto():
    print("\n")
```

```
print ("TASK B.2: Generating Pareto Front (Lift vs Drag)")  
print ("\n")  
  
# Define Weights: 0.0 = Pure Drag Min, 1.0 = Pure Lift Max  
weights = [0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9,  
            1.0]  
  
pareto_points = []  
  
# SCALING FACTOR (Crucial!)  
# Cd is ~0.01, Cl is ~1.0. We multiply Cd by 100 so the  
optimizer treats them equally.  
SCALE_DRAG = 100.0  
  
for w in weights:  
    print (f"\nOptimizing for w={w:.1f} (Lift importance)")  
  
    # Dynamic Objective Function for this specific loop  
    def weighted_objective(x):  
        data = get_airfoil_data(x)  
  
        if not data['valid']:  
            return data['penalty']
```

```
# Objectives

# 1. Maximize Lift -> Minimize (-Cl)
obj_lift = -data['Cl']

# 2. Minimize Drag -> Minimize (Cd * Scale)
obj_drag = data['Cd'] * SCALE_DRAG

# 3. Constraints (Soft Penalty)
penalty_cm = 0
if abs(data['Cm']) > 0.15:
    penalty_cm = (abs(data['Cm']) - 0.15) * 1000

# Weighted Sum
score = (w * obj_lift) + ((1 - w) * obj_drag) +
        penalty_cm
return score

# Run PSO
pso = Particle_Swarm_Optimisation(
    objective_function=weighted_objective,
    dimension=6,
    swarmsize=40, # Smaller swarm is okay for this loop
```

```
c1=1.5, c2=1.5, w_min=0.4, w_max=0.9,
min_bound=[-1.0, -1.0, -1.0, 0.05, 0.05, 0.05], # Your
           bounds
max_bound=[-0.05, -0.05, -0.05, 1.0, 1.0, 1.0]
)

best_pos, best_val, _ = pso.solve(max_generations=30)

# Evaluate the best result to get clean Cl/Cd for plotting
final_data = get_airfoil_data(best_pos)
if final_data['valid']:
    print(f"Result: Cl={final_data['Cl']:.3f}, Cd={
        final_data['Cd']:.5f}")
    pareto_points.append(final_data)
else:
    print("Result: Failed to converge.")

# PLOTTING
cl_vals = [p['Cl'] for p in pareto_points]
cd_vals = [p['Cd'] for p in pareto_points]

plt.figure(figsize=(10, 6))
plt.plot(cd_vals, cl_vals, 'o—', linewidth=2, markersize=8,
```

```
        color='blue')

# Annotate weights
for i, w in enumerate(weights):
    if i < len(cd_vals):
        plt.annotate(f"w={w}", (cd_vals[i], cl_vals[i]),
                      textcoords="offsetpoints", xytext=(0,10)
                      , ha='center')

plt.xlabel('Drag_Coefficient_($C_d$)_ [Minimize]')
plt.ylabel('Lift_Coefficient_($C_l$)_ [Maximize]')
plt.title('Pareto_Front:_Aerodynamic_Efficiency_Trade-off')
plt.grid(True)
plt.savefig("B2_Pareto_Front.png")
plt.close() # Close plot to avoid warning
print("Pareto_plot_saved_to_B2_Pareto_Front.png")

if __name__ == "__main__":
    if not xfoil_worker.find_xfoil():
        print("ERROR:_XFOIL_executable_not_found._Please_place_'_
              xfoil.exe'_ (Windows)_or_'_xfoil'_ (Mac/Linux)_in_this_
              folder.")
    else:
```

```
# run_task_b2_pareto() # Image already exists  
run_task_b3_single_optimization()  
# run_task_b4_parameter_investigation()
```

9.3 Assignment C: Surrogate Model

Listing 4: Assignment C Surrogate Model

```
# Author: John Hoarau and Gemini Pro.  
# Version: 1.0  
  
import numpy as np  
import os  
import time  
import cst  
import xfoil_worker  
import matplotlib.pyplot as plt  
import tempfile  
import shutil  
from AssignmentA import Particle_Swarm_Optimisation  
  
# Configuration  
DESIGN_ALPHA_MEAN = 3.0  
DESIGN_ALPHA_STD = 0.1  
RE = 1000000.0  
N_POINTS = 160
```

```
def get_performance(design_vars, alpha):
    """
    Evaluates Cl, Cd, Cm for a specific Angle of Attack.
    """
    n_vars = len(design_vars)
    mid = n_vars // 2
    wl = design_vars[:mid]
    wu = design_vars[mid:]

    # Generate coordinates
    x, y = cst.cst_airfoil(wl, wu, N=N_POINTS)

    # Save to temp file
    filename = f"temp_c_robust_{time.time_ns()}_{np.random.randint(
        (0,1000))}.dat"
    with open(filename, "w") as f:
        f.write("RobustCandidate\n")
        for i in range(len(x)):
            f.write(f"{x[i]:.6f}{y[i]:.6f}\n")
```

```
# Run XFOIL

try:

    res = xfoil_worker.analyze_airfoil(filename, alpha=alpha,
                                       Re=RE)

except Exception as e:

    print(f"CRITICAL_ERROR: {e}")

    # If it fails, return penalty/None

    res = {'Cl': 0, 'Cd': 1, 'Cm': 0, 'converged': False}


# Clean up

if os.path.exists(filename):

    os.remove(filename)


return res


def calculate_robustness(design_vars, n_samples=100):

    """

    C.1: Compute Mean and StdDev of Cl, Cd, Cm under uncertainty.

    Uncertainty: Alpha ~ N(3.0, 0.1)

    """

    # Sample Alpha

    alphas = np.random.normal(DSIGN_ALPHA_MEAN, DESIGN_ALPHA_STD,
                              n_samples)
```

```
cls = []
cds = []
cms = []
accuracies = [] # To track convergence rate

print(f"Running Robustness Check ({n_samples} samples) ... ")

for i, alpha in enumerate(alphas):
    res = get_performance(design_vars, alpha)

    if res['converged']:
        cls.append(res['Cl'])
        cds.append(res['Cd'])
        cms.append(res['Cm'])
        accuracies.append(1)
    else:
        accuracies.append(0)

# Statistics
stats = {}

if len(cls) > 0:
    stats['Cl_mean'] = np.mean(cls)
```

```
stats['Cl_std'] = np.std(cls)
stats['Cd_mean'] = np.mean(cds)
stats['Cd_std'] = np.std(cds)
stats['Cm_mean'] = np.mean(cms)
stats['Cm_std'] = np.std(cms)
stats['Success_Rate'] = np.mean(accuracies)

else:

    stats['Cl_mean'] = 0
    stats['Cl_std'] = 0
    stats['Cd_mean'] = 1
    stats['Cd_std'] = 0
    stats['Cm_mean'] = 0
    stats['Cm_std'] = 0
    stats['Success_Rate'] = 0.0

return stats


def robust_objective_function(design_vars):
    """
    Objective to minimize: -(Mean Efficiency) + penalty on
    variability?
    Or strictly Reliability Based?
    """
```

```
"""Let's maximize Mean(L/D) while ensuring robust convergence.  
Simple approach: Maximize Mean(Cl)/Mean(Cd)  
"""
```

```
# Let's use 3-Sigma points for efficiency: Mean, Mean+Std,  
Mean-Std  
  
alphas = [DESIGN_ALPHA_MEAN, DESIGN_ALPHA_MEAN +  
          DESIGN_ALPHA_STD, DESIGN_ALPHA_MEAN - DESIGN_ALPHA_STD]  
  
# Robust Objective: Maximize Mean(Efficiency) - k * Std(  
Efficiency)  
  
# This penalizes designs that are unstable across the alpha  
range.
```

```
efficiencies = []
```

```
for alpha in alphas:  
    res = get_performance(design_vars, alpha)  
    if res['converged'] and res['Cd'] > 0:  
        eff = res['Cl'] / res['Cd']  
        efficiencies.append(eff)  
  
    else:
```

```
        # If any point fails , it's not robust

        return 1e9

    mean_eff = np.mean( efficiencies )
    std_eff = np.std( efficiencies )

    k = 1.0 # Weight for robustness
    robust_score = mean_eff - k * std_eff

    return -robust_score


def run_task_c1():
    print( "\n——Task_C.1: Robustness of Optimal Design——" )
    try:
        best_weights = np.load( "best_weights.npy" )
        print( f"Loaded weights from 'best_weights.npy'" )
    except FileNotFoundError:
        print( "CRITICAL: 'best_weights.npy' not found. Skipping Task_C.1. " )
        return

    stats = calculate_robustness( best_weights , n_samples=100 )
```

```
print("\nRobustness_Statistics_(Alpha~N(3.0,0.1)): ")
print(f"Cl:{stats['Cl_mean']:.4f} +/- {stats['Cl_std']:.4f}
")
print(f"Cd:{stats['Cd_mean']:.4f} +/- {stats['Cd_std']:.4f}
")
print(f"ConvergenceRate:{stats['Success_Rate']*100:.1f}%")

# Try importing sklearn
try:
    from sklearn.gaussian_process import GaussianProcessRegressor
    from sklearn.gaussian_process.kernels import Matern,
        WhiteKernel, ConstantKernel
    from sklearn.preprocessing import StandardScaler
    from scipy.stats import qmc
    SKLEARN_AVAILABLE = True
except ImportError:
    print("WARNING: scikit-learn or scipy not found. Surrogate
        modeling will likely fail.")
    SKLEARN_AVAILABLE = False

def generate_doe(n_samples, dimensions=6, bounds=(-0.5, 0.5)):
    """
```

```
##### Generate Design of Experiments using Latin Hypercube Sampling.
##### """
    sampler = qmc.LatinHypercube(d=dimensions, seed=42)
    sample = sampler.random(n=n_samples)

    # Scale to bounds
    lower_bound, upper_bound = bounds
    scaled_sample = qmc.scale(sample, [lower_bound]*dimensions, [
        upper_bound]*dimensions)

    return scaled_sample

def train_surrogate_model(X, y):
    """
    ##### Train a Gaussian Process Regressor.
    ##### """
    # Kernel: Constant * Matern + Noise
    kernel = ConstantKernel(1.0) * Matern(length_scale=1.0, nu
        =2.5) + WhiteKernel(noise_level=1e-5)

    gp = GaussianProcessRegressor(kernel=kernel,
        n_restarts_optimizer=5, normalize_y=True, random_state=42)
    gp.fit(X, y)
```

```
    return gp

def evaluate_surrogate(model, design_vars):
    """
    Predict using the surrogate model.
    Returns: predicted value (negative efficiency)
    """
    # Reshape for single prediction
    X_pred = np.array(design_vars).reshape(1, -1)
    y_pred = model.predict(X_pred)
    return y_pred[0] # Return scalar

def evaluate_surrogate_multi(models, design_vars):
    """
    Wrapper to predict Cl, Cd, Cm and apply constraints.
    """
    X_in = np.array(design_vars).reshape(1, -1)

    # Predict all 3
    cl = models['Cl'].predict(X_in)[0]
    cd = models['Cd'].predict(X_in)[0]
    cm = models['Cm'].predict(X_in)[0]
```

```
# Constraints

# 1. Lift > 0
if cl <= 0: return 1000 + abs(cl)*100

# 2. Drag > 0 (Physically impossible otherwise)
if cd <= 1e-4: return 1000

# 3. Stability |Cm| < 0.15
if abs(cm) > 0.15: return 100 + (abs(cm)-0.15)*1000

# Objective: Maximize Efficiency -> Minimize -L/D
return -(cl/cd)

def run_task_c3_c4():
    if not SKLEARN_AVAILABLE: return

    print("\n——Task_C.3: Multi-Output Surrogate Construction——")

    # 1. Generate Training Data (LHS)
    N_SAMPLES = 30 # Increased to ensure enough valid data
    print(f"Sampling {N_SAMPLES} points via LHS...")

    sampler = qmc.LatinHypercube(d=6, seed=42)
```

```
sample = sampler.random(n=N_SAMPLES)

# CRITICAL FIX: Use Smart Bounds (Same as Task B)
# Lower Weights (indices 0-2): [-1.0, 0.0] -> Curves down
# Upper Weights (indices 3-5): [ 0.0, 1.0] -> Curves up
l_min, l_max = -1.0, 0.05
u_min, u_max = -0.05, 1.0

lower_bounds = [l_min, l_min, l_min, u_min, u_min, u_min]
upper_bounds = [l_max, l_max, l_max, u_max, u_max, u_max]

X_train = qmc.scale(sample, lower_bounds, upper_bounds)

y_cl, y_cd, y_cm = [], [], []
valid_X = []

print("Running_XFOIL_training_batch...")
for i, vars in enumerate(X_train):
    res = get_performance(vars, alpha=DESIGN_ALPHA_MEAN)

    # Check convergence AND physical validity (Drag > 0)
    if res['converged'] and res['Cd'] > 0.0001:
        valid_X.append(vars)
```

```
        y_cl.append(res[ 'Cl' ])
        y_cd.append(res[ 'Cd' ])
        y_cm.append(res[ 'Cm' ])

        print( f"_{i+1}: Valid (Cl={res[ 'Cl' ]:.2f}, Cd
                ={res[ 'Cd' ]:.4f})")

    else:

        print( f"_{i+1}: Invalid/Failed")

if len(valid_X) < 10:

    print( f"CRITICAL: Only {len(valid_X)} valid samples.
           Bounds or XFOIL settings need adjustment.")

return

X_train = np.array(valid_X)
y_cl = np.array(y_cl)
y_cd = np.array(y_cd)
y_cm = np.array(y_cm)

# 2. Train 3 Separate Models

print( f"Training Kriging Models on {len(X_train)} samples...")

gp_models = {
    'Cl': train_surrogate_model(X_train, y_cl),
    'Cd': train_surrogate_model(X_train, y_cd),
```

```
        'Cm': train_surrogate_model(X_train, y_cm)
    }

# 3. Validation
print("\nSurrogate Validation (R2 Scores on Training Data): ")
print(f"Cl Model: {gp_models['Cl'].score(X_train, y_cl):.4f} ")
print(f"Cd Model: {gp_models['Cd'].score(X_train, y_cd):.4f} ")
print(f"Cm Model: {gp_models['Cm'].score(X_train, y_cm):.4f} ")

print("\n——Task C.4: Deterministic Optimization with Surrogate——")

def surrogate_wrapper(x):
    return evaluate_surrogate_multi(gp_models, x)

pso = Particle_Swarm_Optimisation(
    objective_function=surrogate_wrapper,
    dimension=6, swarmsize=40, c1=1.5, c2=1.5,
    w_min=0.4, w_max=0.9, min_bound=lower_bounds, max_bound=
    upper_bounds
```

```
)

best_pos, best_val, _ = pso.solve(max_generations=50)
print(f"Predicted Efficiency: {-best_val:.2f}")

# Verify
print("Verifying result with real XFOIL...")
res = get_performance(best_pos, DESIGN_ALPHA_MEAN)
if res['converged']:
    real_eff = res['Cl']/res['Cd']
    print(f"Actual Efficiency: {real_eff:.2f}")
else:
    print("Verification Failed.")

def run_task_c2():
    print("\n—— Task C.2: Robust/Reliability Based Design Optimization ——")
    print("Running PSO with Robust Objective (Mean Efficiency over 3-Sigma Alphas) ...")

# Robust Optimization Configuration
dimensions = 6
min_bound, max_bound = -0.5, 0.5
```

```
swarm_size = 5 # Small swarm for cost
max_generations = 5 # Very few generations for demo purposes (
    Expensive!)

pso = Particle_Swarm_Optimisation(
    objective_function=robust_objective_function,
    dimension=dimensions,
    swarmsize=swarm_size,
    c1=1.5, c2=1.5, w_min=0.4, w_max=0.9,
    min_bound=min_bound, max_bound=max_bound
)

start_time = time.time()
best_pos_robust, best_val_robust, history = pso.solve(
    max_generations)
end_time = time.time()

print(f"Robust_Optimization_Time:{end_time-start_time:.2f}s
    ")

print(f"Best_Robust_Efficiency:{-best_val_robust:.4f}")

# Verify reliability of this new design
print("Verifying_Robustness_of_RBDO_Design...")
```

```
stats = calculate_robustness(best_pos_robust, n_samples=20)
print(f"Cl: {stats['Cl_mean']:.4f} +/- {stats['Cl_std']:.4f}
      ")
print(f"Cd: {stats['Cd_mean']:.4f} +/- {stats['Cd_std']:.4f}
      ")
print(f"Success_Rate: {stats['Success_Rate']*100:.1f}%")

if __name__ == "__main__":
    run_task_c1()
    # run_task_c2()
    run_task_c3_c4()
```

9.4 CST Parameterization

Listing 5: Class Shape Transformation (CST)

```
# Author: John Hoarau and Gemini Pro.
# Version: 1.0

import numpy as np

def bernstein_poly(n, k, x):
    """
    Calculate the Bernstein polynomial term  $B_{n,k}(x)$ .
    """
    from math import factorial
```

```
    coeff = factorial(n) / (factorial(k) * factorial(n - k))

    return coeff * (x**k) * ((1 - x)**(n - k))


def bernstein_matrix(n, N_points):
    """
    Pre-calculate Bernstein matrix of shape (N_points, n+1).
    """
    theta = np.linspace(0, np.pi, N_points)
    x = 0.5 * (1 - np.cos(theta))

    B = np.zeros((N_points, n + 1))
    for k in range(n + 1):
        B[:, k] = bernstein_poly(n, k, x)
    return x, B


def cst_airfoil(wl, wu, dz=0, N=160):
    """
    Generate airfoil coordinates using CST parameterization(
    Vectorized).
    """
    n_lower = len(wl) - 1
    n_upper = len(wu) - 1
```

```
# Generate/Get Matrix (Could be cached, but fast enough to gen  
here)  
  
# Note: We re-gen x here to avoid passing it around, or we  
call the matrix helper.  
  
x, B_l = bernstein_matrix(n_lower, N)  
  
if n_upper == n_lower:  
    B_u = B_l  
  
else:  
    _, B_u = bernstein_matrix(n_upper, N)  
  
  
C = np.sqrt(x) * (1 - x)  
  
  
# Vectorized Dot Product  
  
S_l = np.dot(B_l, wl)  
S_u = np.dot(B_u, wu)  
  
  
y_l = C * S_l - x * dz / 2.0  
y_u = C * S_u + x * dz / 2.0  
  
  
# Upper surface: x=1 to 0  
  
x_u_pts = np.flip(x)  
y_u_pts = np.flip(y_u)
```

```
# Lower surface: x=0 to 1
x_l_pts = x[1:]
y_l_pts = y_l[1:]

x_combined = np.concatenate([x_u_pts, x_l_pts])
y_combined = np.concatenate([y_u_pts, y_l_pts])

return x_combined, y_combined

def calculate_thickness_distribution(wl, wu, N=160):
    """
    Calculate the thickness distribution (upper and lower surfaces)
    separately.

    Returns x, y_upper, y_lower

    """
    theta = np.linspace(0, np.pi, N)
    x = 0.5 * (1 - np.cos(theta))

    C = np.sqrt(x) * (1 - x)

    n_lower = len(wl) - 1
    n_upper = len(wu) - 1
```

```
S_l = np.zeros_like(x)
S_u = np.zeros_like(x)

for i in range(len(wl)):
    S_l += wl[i] * bernstein_poly(n_lower, i, x)

for i in range(len(wu)):
    S_u += wu[i] * bernstein_poly(n_upper, i, x)

y_l = C * S_l
y_u = C * S_u

return x, y_u, y_l

if __name__ == "__main__":
    # Test
    # NACA 0012 approx weights (approximate)
    # It's not exact, but fits.
    # For a generic test, let's use some dummy weights.
    wu = [0.2, 0.2, 0.2]
    wl = [-0.2, -0.2, -0.2] # Lower weights usually negative in
        # basic CST formulation or handle sign in formula?
    # Actually, in many CST implementations, S(x) represents the
```

*shape, and we usually define $y = C * S$.*

If S is positive, y is positive. For lower surface, we ideally want negative y.

*# The formula $y_l = C * S_l$ used above implies S_l should be negative for lower surface coordinates?*

*# Or we apply the sign explicitly: $y_{lower} = - C * S_{lower}$?*

*# Let's stick to the formula: $y = C(x) * S(x)$. So weights for lower surface should result in negative values.*

Thus wl elements should likely be negative.

```
x, y = cst_airfoil(wl, wu)

# Write to file to check
with open("cst_test.dat", "w") as f:
    f.write("CST_Test_Airfoil\n")
    for i in range(len(x)):
        f.write(f"{x[i]:.6f} {y[i]:.6f}\n")
print("Generated cst_test.dat")
```

9.5 XFOIL Worker

Listing 6: XFOIL Worker Interface

```
# Author: John Hoarau and Gemini Pro.
# Version: 3.0 (Windows Path Fix)
```

```
import os
import subprocess
import shutil
import sys

def find_xfoil():
    """Find_XFOIL_executable."""
    base_dir = os.path.dirname(os.path.abspath(__file__))
    # Check common locations
    paths = [
        os.path.join(base_dir, "XFOIL6.99", "xfoil.exe"),
        os.path.abspath("xfoil.exe"),
        os.path.abspath("XFOIL6.99/xfoil.exe"),
    ]
    for p in paths:
        if os.path.exists(p): return p
    if shutil.which("xfoil.exe"): return shutil.which("xfoil.exe")
    return None

def analyze_airfoil(coords_file, alpha=0, Re=1e6, iter_limit=100,
xfoil_path=None):
    """Run_XFOIL_handling_Windows_paths_with_spaces."""
    if xfoil_path is None:
```

```
xfoil_path = find_xfoil()

if xfoil_path is None:
    # Fallback for Windows if not found in path
    raise FileNotFoundError("XFOIL_executable_not_found.
                             Please_put_xfoil.exe_in_this_folder.")

# Get absolute paths
abs_coords_file = os.path.abspath(coords_file)
work_dir = os.path.dirname(abs_coords_file)
file_name_only = os.path.basename(abs_coords_file)

polar_file = f"polar_{file_name_only}.txt"
abs_polar_file = os.path.join(work_dir, polar_file)

# cleanup previous polar
if os.path.exists(abs_polar_file):
    os.remove(abs_polar_file)

# Command Sequence
# CRITICAL FIX: We run subprocess inside 'work_dir' so we don't
send paths with spaces to XFOIL
input_cmds = [
    "PLOP", "G", "", # Disable graphics
```

```
f "LOAD_{file_name_only}" ,  
"PANE" ,  
"OPER" ,  
f "Visc_{Re}" ,  
f "ITER_{iter_limit}" ,  
"PACC" ,  
polar_file ,           # Output file  
" ,                   # Confirm "Overwrite?" if it asks  
"ALFA_{0.0}" ,         # Init wake  
f "ALFA_{alpha}" ,     # Run target  
"PACC" ,  
"QUIT"  
]
```

```
stdin_data = "\n".join(input_cmds)
```

```
try:
```

```
    # Run XFOIL setting cwd to the data folder
```

```
    process = subprocess.Popen(  
        [xfoil_path] ,
```

```
        stdin=subprocess.PIPE ,
```

```
        stdout=subprocess.PIPE ,
```

```
        stderr=subprocess.PIPE ,
```

```
        text=True,
        cwd=work_dir,  # <— MAGIC FIX for OneDrive/Spaces
        shell=False
    )
    stdout, stderr = process.communicate(input=stdin_data,
                                         timeout=15)

except subprocess.TimeoutExpired:
    process.kill()
    # print("Debug: XFOIL Timed Out") # Uncomment to debug
    return {'Cl': 0, 'Cd': 1, 'Cm': 0, 'converged': False}
except Exception as e:
    print(f"XFOIL_Error: {e}")
    return {'Cl': 0, 'Cd': 1, 'Cm': 0, 'converged': False}

# Parse Results
if not os.path.exists(abs_polar_file):
    return {'Cl': 0, 'Cd': 1, 'Cm': 0, 'converged': False}

try:
    data = parse_polar(abs_polar_file, target_alpha=alpha)
    # cleanup
    try:
```

```
        os.remove(abs_polar_file)

    except PermissionError:

        pass # sometimes file is locked, ignore

    return data

except Exception:

    return {'Cl': 0, 'Cd': 1, 'Cm': 0, 'converged': False}

def parse_polar(polar_file, target_alpha):

    results = {'Cl': 0, 'Cd': 1, 'Cm': 0, 'converged': False}

    try:

        with open(polar_file, 'r') as f:

            lines = f.readlines()

        for line in lines:

            parts = line.split()

            if len(parts) < 5: continue

            try:

                curr_alpha = float(parts[0])

                if abs(curr_alpha - target_alpha) < 0.01:

                    results['Cl'] = float(parts[1])

                    results['Cd'] = float(parts[2])

                    results['Cm'] = float(parts[4])

                    results['converged'] = True
```

```
        return results
    except ValueError:
        continue
except Exception:
    pass
return results
```